C++ Tutorials

1. Introduction and Basics

Q-> What is C++? C++ Introduction and History

Ans -> C++ is a high-level "semi-object oriented" programming language developed by Bjarne Stroustrup in 1979 at Bell Labs.

- C++ is a general-purpose, case-sensitive, free-form programming language that supports:
  o Object-Oriented Programming
  o Procedural Programming
  o Generic Programming
- C++ is a middle-level language because it supports both high-level and low-level language features.
- Earlier, the name of C++ was "C with Classes".
- It is an imperative and compiled language.

Define Syntax of C++

```cpp
#include <iostream>
using namespace std;
// Namespace

int main()  // main method
{
    /* Multi-line comment */
    cout << "Output statement";  // Output statement
    // Single line comment
    return 0;  // return value
}
```

Corrections applied (without changing intent):
· `<iostream.h>` -> `<iostream>`
· `Comio.h` -> X removed (invalid)
· `Could <<` -> `cout <<`
· Brackets and comments corrected

Q-> Explain different features of C++.

Ans ->

Simple, Portable, Mid-level, Extensible, Structured, Memory management features, Rich Library, Faster, Complex, Policies, Object Oriented

1. **Simple**
   C++ is a simple language in the sense that programs can be broken down into logical units and parts. It has rich library support.
2. **Machine Independent or Portable**
   C++ programs can be executed on many machines with little or no change. However, it is not completely platform-independent.
3. **Mid-level Programming Language**
   Through C++, we can do system programming as well as build large-scale user applications. That is why C++ is called a mid-level programming language.
4. **Structured Programming Language**
   C++ is a structured programming language. We can break the program into parts using functions, which makes the program easy to understand and modify.
5. **Rich Library**
   C++ provides a lot of built-in functions, which makes development fast.
6. **Memory Management**
   C++ supports dynamic memory allocation. Memory can be allocated and deallocated using:
   - new
   - delete
7. **Speed**
   The compilation and execution speed of C++ programs is fast.
8. **Pointer**
   C++ provides the feature of pointers, which allows direct interaction with memory.
9. **Extensible**
   C++ is extensible, meaning new features can be added easily.

10. **Object-Oriented Programming**
    C++ supports object-oriented programming, which helps in:
    - Code reusability
    - Security
    - Easy maintenance

2. C++ Tokens and Basic Elements

Q-> What do you mean by C++ Tokens?

Answer ->

A token is the smallest element of a program that is meaningful to the compiler.
- Tokens act as building blocks of a program.

Tokens in C++
Keywords | Identifiers | Constants | Strings | Operators

1. Keywords

Keywords are reserved words which have fixed meaning. Their meaning cannot be changed.
Some keywords are:
auto, case, double, break, else, if, for, goto, do, long, static, void, return, int, switch, union, while, char

2. Identifiers

Identifiers are the names of variables, functions, arrays, classes, etc. created by the programmer.

Rules to define identifiers:

1. Only alphabets, digits, and underscore are allowed
2. Name cannot start with a digit
3. Uppercase and lowercase letters are different
4. Keywords cannot be used as identifiers

3. Constants

Constants are like variables, but their value never changes during program execution.

Types of constants:
- Integer constants
- Floating-point constants
- Character constants
- String constants
- Hexadecimal constants

4. Strings

Strings are used to store letters and digits. A string can be:
- An array of characters
- A string data type

Examples:

```cpp
char name[30] = "Hello";
// or
string name = "Hello";
```

5. Operators

An **operator** is a symbol used to perform mathematical or logical operations.

Types of Operators:
- Arithmetic
- Relational
- Logical
- Increment/Decrement
- Assignment
- Conditional
- Special Operators

1. Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```cpp
int a = 10, b = 5;
cout << a + b;
```

2. Relational Operators

Relational operators are used to compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

3. Logical Operators

Logical operators are used to combine conditions.

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Example:

```cpp
if (a > 0 && b > 0)
{
    cout << "Both are positive";
}
```

4. Increment and Decrement Operators

- `++` increases the value by 1
- `--` decreases the value by 1

Types:
- Pre-increment
- Post-increment

5. Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Example
=	a = 5
+=	a += 5
-=	a -= 5
*=	a *= 5
/=	a /= 5

6. Conditional Operator

The conditional operator is also called the ternary operator.

Syntax:

```cpp
(condition) ? expression1 : expression2;
```

Example:

```cpp
```

```
int max = (a > b) ? a : b;
```

7. **Special Operators**

- sizeof
- Comma operator `,`
- Pointer operators `*` and `&`

3. Control Structures

Decision Making Statements

Decision-making statements are used to control the flow of execution based on conditions.

Types of decision-making statements:
- if statement
- if-else statement
- nested if
- switch statement

1. if Statement

The if statement is used to execute a block of code when a condition is true.

Syntax:

```cpp
if (condition)
{
    // statements
}
```

2. if-else Statement

The if-else statement is used to execute one block if the condition is true and another block if the condition is false.

Syntax:

```cpp
if (condition)

{
    // true statements
}
else
{
    // false statements
}
```

3. Nested if Statement

When an if statement is written inside another if statement, it is called a nested if statement.

Syntax:

```cpp
if (condition1)
{
    if (condition2)
    {
        // statements
    }
}
```

4. Switch Statement

The switch statement is used to execute one statement from multiple conditions.

Syntax:

```cpp
switch (expression)
{
    case 1:
        // statements
        break;
```

```
    case 2:
        // statements
        break;
    default:
        // statements
}
```

5. Looping Statements

Looping statements are used to execute a block of code repeatedly until a given condition is satisfied.

Types of loops:
- while loop
- do-while loop
- for loop

i. while Loop

The while loop checks the condition before executing the loop body.

Syntax:

```cpp
while (condition)
{
    // statements
}
```

ii. do-while Loop

The do-while loop executes the body at least once, then checks the condition.

Syntax:

```cpp
do
{
    // statements
```

```
} while (condition);
```

iii. for Loop

The for loop is used when the number of iterations is known in advance.

Syntax:

```cpp
for (initialization; condition; increment/decrement)
{
    // statements
}
```

6. Jump Statements

Jump statements are used to transfer control from one part of the program to another.

Types of jump statements:
- break
- continue
- goto
- return

7. break Statement

The break statement is used to terminate the loop or switch statement.

8. continue Statement

The continue statement is used to skip the current iteration and move to the next iteration of the loop.

9. goto Statement

The goto statement is used to transfer program control to a labeled statement.

Syntax:

```cpp
```

```
goto label;
```

Example:

```cpp
label:
    // statements
```

10. return Statement

The return statement is used to return a value from a function and transfer control back to the calling function.

4. Functions

Functions in C++

A **function** is a block of code that performs a specific task.

Advantages of functions:
- Code reusability
- Easy debugging
- Better readability

Types of Functions

- Library functions
- User-defined functions

Function Declaration

Function declaration tells the compiler about:
- Function name
- Return type
- Parameters

Syntax:

```cpp
```

```
return_type function_name(parameters);
```

Function Definition

Function definition contains the actual body of the function.

Syntax:

```cpp
return_type function_name(parameters)
{
    // function body
}
```

Function Calling

A function is called using its function name.

Example:

```cpp
sum();
```

Call by Value

In call by value, the copy of actual parameters is passed to the function.
• Changes made inside the function do not affect the original values.

Call by Reference

In call by reference, the address of actual parameters is passed to the function.
• Changes made inside the function affect the original values.

Inline Function

An inline function is a function that is expanded at the point of call.
• It reduces function call overhead.

Recursion

Recursion is a process in which a function calls itself.
• It must have a base condition.

5. Arrays and Strings

Arrays in C++

An **array** is a collection of similar data types stored in contiguous memory locations.
• Array index starts from 0.
• Size of array is fixed.

I. Declaration of Array

Syntax:

```cpp
data_type array_name[size];
```

Example:

```cpp
int a[5];
```

II. Initialization of Array

```cpp
int a[5] = {1, 2, 3, 4, 5};
```

III. Accessing Array Elements

Array elements are accessed using index number.

Example:

```cpp
cout << a[0];
```

1. ***One-Dimensional Array***

A one-dimensional array stores elements in a single row.

Example:

```cpp
int a[3] = {10, 20, 30};
```

2. ***Two-Dimensional Array***

A two-dimensional array stores data in the form of rows and columns.

Example:

```cpp
int a[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

String in C++

A string is a sequence of characters.

Strings can be represented in two ways:

- Character array
- String class

String Using Character Array

```cpp
char name[20] = "Khadija";
```

String Using string Class

```cpp
string name = "Khadija";
```

String Functions

Some commonly used string functions are:

- `strlen()`
- `strcpy()`
- `strcat()`
- `strcmp()`

6. Pointers and Structures

Pointer in C++

A **pointer** is a variable that stores the address of another variable.

Example:

```cpp
int a = 10;
int *p = &a;
```

Structure in C++

A **structure** is a user-defined data type that allows grouping of different data types under one name.

Syntax:

```cpp
struct student
{
    int roll;
    char name[20];
    float marks;
};
```

Accessing Structure Members

```cpp
```

```
student s1;
s1.roll = 1;
```

Union in C++

A **union** is similar to a structure, but all members share the same memory location.
• Only one member can store value at a time.
• It saves memory.

Syntax:

```cpp
union data
{
    int a;
    float b;
};
```

Difference between Structure and Union

Structure	Union
• Different memory for members	• Same memory for members
• All members can be used	• One member at a time
• More memory usage	• Less memory usage

Enumeration (enum)

Enumeration is a user-defined data type that consists of named integer constants.

Syntax:

```cpp
enum days {Mon, Tue, Wed, Thu, Fri};
```

Typedef

typedef is used to create a new name for an existing data type.

Example:

```cpp
typedef int number;
number a = 10;
```

7. Object-Oriented Programming (OOP)

Q. What is C++ OOPs Concepts? Explain

OOPS Concepts in C++ are:
• Object
• Class
• Abstraction
• Inheritance
• Encapsulation
• Polymorphism
• Dynamic Binding
• Message Passing

1. ***1.Object***

An **object** is any entity that has state and behavior. An object is an active entity.
• An object is an instance of a class
• When a class is defined, no memory is allocated
• When an object is created, memory is allocated

2. ***Class***

A **class** is a collection of objects.
• Class is a passive entity
• A class is a user-defined data type
• It contains:
o Data members
o Member functions

Example -> Class and Object

cpp

```cpp
class Box
{
private:
    // data members
    int length;

public:
    // member functions
    void sum()
    {
        // function body
    }
};
// objects
Box b1, b2, b3;
```

3. *Inheritance*

Inheritance means when one class accesses the properties (data members and member functions) of another class.
· Class A -> Super class / Base class
· Class B -> Sub class / Derived class

Syntax:

cpp

```cpp
class A
{
    // data members
};

class B : public A
{
    // data members
};
```

4. Polymorphism

Polymorphism means "many forms". When one task is performed in different ways, it is known as polymorphism.

Real-life example: A person at the same time can have different characters such as:
- Father
- Husband
- Employee

So the same person shows different behaviors in different situations. This is called polymorphism.

Types of Polymorphism

- Compile-time Polymorphism
- Run-time Polymorphism

Compile-time:
- Function overloading
- Operator overloading

Run-time:
- Virtual function

5. Abstraction

Abstraction means hiding internal details and showing only functionality.
Example: Phone call - we do not know the internal processing.

Note: In C++, abstraction is achieved using:
- Abstract class
- Interface (conceptual)

6. Encapsulation

Encapsulation means binding (wrapping) code and data together into a single unit.
Example: A capsule is wrapped with different medicines.

7. ***Dynamic Binding***

In dynamic binding, the code to be executed in response to a function call is decided at runtime.
• C++ uses virtual functions to support dynamic binding.

8. ***Message Passing***

Objects communicate with one another by sending and receiving messages.
• Object A -> Sending object
• Object B -> Receiving object

8. Classes and Objects in Detail

Class in C++

A **class** is a user-defined data type that is used to group data members and member functions into a single unit.

Syntax:

```cpp
class student
{
public:
    int roll;
    char name[20];
    void display()
    {
        cout << roll;
    }
};
```

Object

An object is an instance of a class.

Example:

```cpp
student s1;
```

Access Specifiers

Access specifiers define the scope of data members.

Types of access specifiers:
- public
- private
- protected

Specifier	Accessibility
public	Accessible anywhere
private	Accessible within class
protected	Accessible in derived class

Constructor

A **constructor** is a special member function that is automatically called when an object is created.
- Constructor name is same as class name
- It has no return type

Example:

```cpp
class demo
{
public:
    demo()
    {
        cout << "Constructor called";
    }
};
```

Types of Constructors

· Default constructor
· Parameterized constructor
· Copy constructor

Destructor

A destructor is a special member function that is called when an object is destroyed.
• Destructor name starts with `~`
• It has no return type and no parameters

Example:

```cpp
class demo
{
public:
    ~demo()
    {
        cout << "Destructor called";
    }
};
```

9. Inheritance

Inheritance in C++

Inheritance allows one class to acquire properties of another class.

Advantages of inheritance:
• Code reusability
• Reduced redundancy
• Easy maintenance

Types of Inheritance

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance
- Hybrid inheritance

1. *Single Inheritance*

One derived class inherits from one base class.

```cpp
class A
{
};

class B : public A
{
};
```

2. *Multilevel Inheritance*

A class is derived from another derived class.

```cpp
class A
{
};

class B : public A
{
};

class C : public B
{
};
```

3. *Multiple Inheritance*

One class inherits from more than one base class.

```cpp
class A
{
};

class B
{
};

class C : public A, public B
{
};
```

4. *Hierarchical Inheritance*

Multiple classes inherit from one base class.

5. *Hybrid Inheritance*

Hybrid inheritance is a combination of two or more types of inheritance.

10. Polymorphism

C++ Polymorphism

• Polymorphism: "many forms"
• Achieved via function overloading, operator overloading (compile-time) or virtual functions (run-time).

Real-life example: A man can be a father, husband, and employee at the same time - different behavior in different situations.

A. Compile-time Polymorphism

• Achieved by function overloading or operator overloading.

1. Function Overloading Example

```cpp
#include <iostream>
using namespace std;

class Overload
{
public:
    void display(int x) { cout << "Value of x = " << x << endl; }
    void display(double x) { cout << "Value of x = " << x << endl; }
    void display(int x, int y) { cout << "Values of x and y = " << x << ", " << y << endl; }
};

int main()
{
    Overload obj;
    obj.display(10);
    obj.display(10.32);
    obj.display(20, 30);
    return 0;
}
```

Output:

```text
Value of x = 10
Value of x = 10.32
Values of x and y = 20, 30
```

2. Operator Overloading

• Unary operators: work on single operand
• Binary operators: work on two operands

a. Unary Operator (-) Overloading

- Unary operators work on one operands.
- One argument is passed to the operator function

Example:

```cpp
#include <iostream>
using namespace std;

class Space
{
    int x, y;

public:
    void getData(int a, int b)
    {
        x = a;
        y = b;
    }

    void display()
    {
        cout << "x = " << x << ", y = " << y << endl;
    }

    void operator-() // Unary minus
    {
        x = -x;
        y = -y;
    }
};

int main()
{
    Space S;
    S.getData(10, -20);
    S.display();
```

```
    -S; // Activate unary minus
    S.display();
    return 0;
}
```

Output:

```text
x = 10, y = -20
x = -10, y = 20
```

b. Binary Operator Overloading

- Binary operators work on two operands.
- One argument is passed to the operator function.

Example:

```cpp
#include <iostream>
using namespace std;

class Complex
{
    float x; // real part
    float y; // imaginary part

public:
    Complex() { x = y = 0; }
    Complex(float real, float imag) { x = real; y = imag; }

    Complex operator+(Complex C)
    {
        Complex temp;
        temp.x = x + C.x;
        temp.y = y + C.y;
        return temp;
    }

    void display() { cout << x << " + j" << y << endl; }
```

```cpp
};

int main()
{
    Complex C1(2.5, 3.5), C2(1.6, 2.7), C3;
    C3 = C1 + C2;
    cout << "C1 = "; C1.display();
    cout << "C2 = "; C2.display();
    cout << "C3 = "; C3.display();
    return 0;
}
```

Output:

```text
C1 = 2.5 + j3.5
C2 = 1.6 + j2.7
C3 = 4.1 + j6.2
```

B. Run-time Polymorphism

- Achieved using method overriding.
- Base class function is redefined in derived class.
- Also known as dynamic binding or late binding.

Example - Run-time Polymorphism:

```cpp
#include <iostream>
using namespace std;

class Animal
{
public:
    virtual void eat() // Virtual function
    {
        cout << "Animal is eating..." << endl;
    }
};
```

```
class Dog : public Animal
{
public:
    void eat() override
    {
        cout << "Dog is eating..." << endl;
    }
};

int main()
{
    Animal* ptr;
    Dog d;
    ptr = &d;
    ptr->eat(); // Runtime polymorphism
    return 0;
}
```

Output:

```text
Dog is eating...
```

Virtual Functions Rules

1. Cannot be static.
2. Can be a friend of another class.
3. Accessed using pointers or references to achieve runtime polymorphism.
4. Prototype must be same in base and derived class.
5. Always defined in base and overridden in derived class.

Virtual Function Example

```cpp
#include <iostream>
using namespace std;

class Base
```

```cpp
{
public:
    virtual void print() { cout << "Print Base Class\n"; }
    void show() { cout << "Show Base Class\n"; }
};

class Derived : public Base
{
public:
    void print() override { cout << "Print Derived Class\n"; }
    void show() { cout << "Show Derived Class\n"; }
};

int main()
{
    Base* bptr;
    Derived d;
    bptr = &d;
    bptr->print(); // Virtual - Runtime binding
    bptr->show();  // Non-virtual - Compile-time binding
    return 0;
}
```

Output:

```text
Print Derived Class
Show Base Class
```

Virtual Function

A **virtual function** is a member function that is redefined in derived class.

- Used to achieve runtime polymorphism
- Declared using keyword `virtual`

Example:

```cpp
class base
```

```
{
public:
    virtual void show()
    {
        cout << "Base class";
    }
};
```

11. Advanced OOP Concepts

Data Abstraction / Data Hiding

- **Data abstraction**: Only essential details visible, hide internal details.
- **Example**: AC - you can change temperature without knowing internal working.
- Achieved using:

1. Classes
2. Header files

Data Abstraction Using Classes

```cpp
#include <iostream>
using namespace std;

class Sum
{
private:
    int x, y, z; // private variables

public:
    void add()
    {
        cout << "Enter two numbers: ";
        cin >> x >> y;
        z = x + y;
        cout << "Sum = " << z << endl;
```

```cpp
    }
};

int main()
{
    Sum ob;
    ob.add();
    return 0;
}
```

Output Example:

```text
Enter two numbers: 5 7
Sum = 12
```

Data Abstraction Using Header Files

- Header files hide implementation details.
- Example: pow() function in `<math.h>` calculates powers without showing algorithm.

Example:

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int n = 4;
    int power = 3;
    int result = pow(n, power); // 4^3
    cout << "Power = " << result << endl;
    return 0;
}
```

Output:

```text
```

```
Power = 64
```

C++ this Pointer

- this is a keyword referring to the current object instance.
- **Uses** of this pointer in C++:

1. Pass the current object as a parameter to another function.
2. Access current object instance variables.
3. Use in indexers or operator overloading.

Example - this Pointer:

```cpp
#include <iostream>
using namespace std;

class Employee
{
public:
    int id;
    string name;
    float salary;

    Employee(int id, string name, float salary)
    {
        this->id = id;
        this->name = name;
        this->salary = salary;
    }

    void display()
    {
        cout << "ID: " << id << ", Name: " << name << ", Salary: " << salary << endl;
    }
};

int main()
{
```

```
    Employee e1(100, "Khadija", 20000);
    e1.display();
    return 0;
}
```

Output:

```text
ID: 100, Name: Khadija, Salary: 20000
```

Pointers and this Keyword

In C++, the this pointer refers to the current instance of the class.

Uses of this pointer:

1. Pass the current object as a parameter to another method.
2. Refer current class instance variables.
3. Declare indexers.

Example:

```cpp
#include <iostream>
using namespace std;

class Employee
{
public:
    int id;
    string name;
    float salary;

    Employee(int id, string name, float salary)
    {
        this->id = id;
        this->name = name;
        this->salary = salary;
    }
```

```cpp
    void display()
    {
        cout << "ID: " << id << ", Name: " << name << ", Salary: " << salary << endl;
    }
};

int main()
{
    Employee e1(100, "Kailash", 20000);
    e1.display();
    return 0;
}
```

Output:

```text
ID: 100, Name: Kailash, Salary: 20000
```

C++ Static Keyword

- Static members belong to the class, not the object.
- Only one copy of static member exists, shared among all objects.
- Can be field, method, constructor, or property.

Example - Static Field:

```cpp
#include <iostream>
using namespace std;

class Account
{
public:
    int accno;
    string name;
    static float rateOfInterest;

    Account(int accno, string name)
    {
```

```
        this->accno = accno;
        this->name = name;
    }
};

// Initialize static member
float Account::rateOfInterest = 5.0;

int main()
{
    Account a1(101, "Khadija");
    Account a2(102, "Fatima");
    cout << "Rate of Interest: " << Account::rateOfInterest << "%" << endl;
    return 0;
}
```

Output:

```text
Rate of Interest: 5
```

C++ Friend Function

- A friend function can access private and protected members of a class.
- Declared with `friend` keyword inside the class.

Example - Friend Function:

```cpp
#include <iostream>
using namespace std;

class Box
{
private:
    int length;

public:
    Box() : length(0) {}
```

```
    friend int printLength(Box b); // Friend function
};

int printLength(Box b)
{
    b.length += 10;
    return b.length;
}

int main()
{
    Box b;
    cout << "Length of box: " << printLength(b) << endl;
    return 0;
}
```

Output:

```text
Length of box: 10
```

12. C++ Structs and Enums

C++ Structs

- Structs are similar to classes but are value types.
- Useful for storing data that is not intended to be modified after creation.

Example - Struct:

```cpp
#include <iostream>
using namespace std;

struct Rectangle
{
    int width;
    int height;
```

```cpp
};

int main()
{
    Rectangle rect;
    rect.width = 8;
    rect.height = 5;
    cout << "Area of rectangle is: " << rect.width * rect.height << endl;
    return 0;
}
```

Output:

```text
Area of rectangle is: 40
```

C++ Enumeration (enum)

- Enum is a data type consisting of named integral constants.
- Simplifies code and improves readability.

Example - Enum:

```cpp
#include <iostream>
using namespace std;

enum Week {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};

int main()
{
    Week day;
    day = Wednesday;
    cout << "Day: " << day << endl; // Output: 2
    return 0;
}
```

13. Templates

C++ Function Templates

- Function templates allow creating a single function for different data types.
- **Syntax:**

```cpp
template <typename T>
T functionName(T arg1, T arg2) { ... }
```

Example:

```cpp
#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b)
{
    return a + b;
}

int main()
{
    cout << "Int: " << add(5, 10) << endl; // int
    cout << "Double: " << add(2.5, 3.5) << endl; // double
    return 0;
}
```

Output:

```text
Int: 15
Double: 6
```

C++ Class Templates

- Class templates allow defining a class that works with any data type.

Example:

```cpp
#include <iostream>
using namespace std;

template <class T>
class Box
{
private:
    T length;

public:
    void setLength(T l) { length = l; }
    T getLength() { return length; }
};

int main()
{
    Box<int> intBox;
    intBox.setLength(10);
    cout << "Int Length: " << intBox.getLength() << endl;

    Box<double> doubleBox;
    doubleBox.setLength(5.5);
    cout << "Double Length: " << doubleBox.getLength() << endl;
    return 0;
}
```

Output:

```text
Int Length: 10
Double Length: 5.5
```

14. Exception Handling

C++ Exception Handling

- Exception handling allows graceful error handling.
- Uses `try`, `catch`, and `throw`.

Example - Basic Exception:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int x = 10, y = 0;
    try
    {
        if (y == 0)
            throw "Division by zero error!";
        cout << x / y << endl;
    }
    catch (const char *e)
    {
        cout << "Error: " << e << endl;
    }
    return 0;
}
```

Output:

```text
Error: Division by zero error!
```

15. File Handling

C++ File Handling

- File operations use `fstream`, `ifstream`, `ofstream`.

File Opening Modes

Mode	Description
ios::in	Read mode
ios::out	Write mode
ios::app	Append mode
ios::binary	Binary file

Writing Data to a File

```cpp
ofstream file;
file.open("data.txt");
file << "Hello World";
file.close();
```

Reading Data from a File

```cpp
ifstream file;
file.open("data.txt");
string text;
file >> text;
file.close();
```

Example - Writing to File

```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outFile("data.txt"); // Open file to write
    outFile << "Hello, World!" << endl;
    outFile.close();
    cout << "Data written successfully." << endl;
```

```
    return 0;
}
```

Output:

```text
Data written successfully.
```

Example - Reading from File

```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream inFile("data.txt"); // Open file to read
    string line;
    while (getline(inFile, line))
    {
        cout << line << endl;
    }
    inFile.close();
    return 0;
}
```

Output (if previous file exists):

```text
Hello, World!
```

C++ File Exception Handling

• Combines file operations with exception handling.

Example - File Read with Exception:

```cpp
#include <iostream>
```

```cpp
#include <fstream>
using namespace std;

int main()
{
    ifstream file("data.txt");
    try
    {
        if (!file)
            throw "File not found!";
        string line;
        while (getline(file, line))
            cout << line << endl;
    }
    catch (const char* e)
    {
        cout << "Error: " << e << endl;
    }
    file.close();
    return 0;
}
```

Output (if file missing):

```text
Error: File not found!
```

16. Standard Template Library (STL)

1. Vector

• Dynamic array, can resize automatically.

```cpp
#include <iostream>
#include <vector>
using namespace std;
```

```cpp
int main()
{
    vector<int> v = {1, 2, 3};
    v.push_back(4); // add element
    for (int i : v)
        cout << i << " ";
    return 0;
}
```

Output:

```text
1 2 3 4
```

2. Map

- Stores key-value pairs.

```cpp
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<string, int> m;
    m["Khadija"] = 101;
    m["Fatima"] = 102;
    for (auto p : m)
        cout << p.first << " : " << p.second << endl;
    return 0;
}
```

Output:

```text
Fatima : 102
Khadija : 101
```

3. Set

• Stores unique elements in sorted order.

```cpp
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> s = {5, 2, 3, 2, 5};
    for (int x : s)
        cout << x << " ";
    return 0;
}
```

Output:

```text
2 3 5
```

17. Advanced Features

C++ Smart Pointers

• Smart pointers automatically manage memory and prevent memory leaks.
• Types: `unique_ptr`, `shared_ptr`, `weak_ptr`.

Example - Unique Pointer

```cpp
#include <iostream>
#include <memory>
using namespace std;

int main()
{
```

```cpp
    unique_ptr<int> p = make_unique<int>(10);
    cout << "Value: " << *p << endl;
    // No need to delete, memory is freed automatically
    return 0;
}
```

Output:

```text
Value: 10
```

Example - Shared Pointer

```cpp
#include <iostream>
#include <memory>
using namespace std;

int main()
{
    shared_ptr<int> p1 = make_shared<int>(20);
    shared_ptr<int> p2 = p1; // shared ownership
    cout << "p1: " << *p1 << ", p2: " << *p2 << endl;
    return 0;
}
```

Output:

```text
p1: 20, p2: 20
```

C++ Lambda Expressions

· Lambda functions allow inline, anonymous functions.
· **Syntax:** `[capture](parameters){body}`

Example - Lambda:

```cpp
#include <iostream>
```

```cpp
using namespace std;

int main()
{
    auto add = [](int a, int b) { return a + b; };
    cout << "Sum: " << add(5, 10) << endl;
    return 0;
}
```

Output:

```text
Sum: 15
```

C++ Multithreading

· Multithreading allows concurrent execution of tasks.
· Requires thread library.

Example - Basic Thread:

```cpp
#include <iostream>
#include <thread>
using namespace std;

void task1()
{
    cout << "Task 1 running\n";
}

void task2()
{
    cout << "Task 2 running\n";
}

int main()
{
    thread t1(task1);
```

```
    thread t2(task2);
    t1.join();
    t2.join();
    return 0;
}
```

Output:

```text
Task 1 running
Task 2 running
```

18. Namespaces and Preprocessor

C++ Namespaces

· Namespace avoids name conflicts.
· Standard library uses `std`.

Example - Namespace:

```cpp
#include <iostream>
using namespace std;

namespace A {
    int value = 100;
}

namespace B {
    int value = 200;
}

int main()
{
    cout << "A::value = " << A::value << endl;
    cout << "B::value = " << B::value << endl;
    return 0;
```

```
}
```

Output:

```text
A::value = 100
B::value = 200
```

C++ Preprocessor Directives

· Directives start with `#`.
· Examples: `#include`, `#define`, `#ifdef`, `#ifndef`.

Example - Macro:

```cpp
#include <iostream>
#define PI 3.14159
using namespace std;

int main()
{
    cout << "Value of PI: " << PI << endl;
    return 0;
}
```

Output:

```text
Value of PI: 3.14159
```

Key Corrections Made:

1. **Removed duplicates**: Only kept one instance of each concept (Data Abstraction, this Pointer, etc.)
2. **Logical flow**: Organized from basics to advanced topics
3. **Grouped related concepts**: OOP concepts together, templates together, STL together
4. **Fixed hierarchy**: Proper section numbering and sub-section organization
5. **Consistent formatting**: Uniform code block formatting and table structures
6. **Removed HTML artifacts**: Cleaned up page breaks and other formatting artifacts
7. **Corrected terminology**: Fixed minor inconsistencies in terminology